

★ WEEK 5 · PROBLEM SET

Finance Applications: Three Hands-On Problems

Logistic by Hand · KNN Vote · Naive Bayes Screening

Each problem: Statement → Principles & Lab Guide → Hand Calculation → Claude Code Prompt

Small numbers you can compute with a pencil — then a metaprompt that lets Claude Code scale the same idea to real data. Hand first, machine second.

Every Problem Follows the Same Four Steps

Work top to bottom — do not skip the hand calculation, it is where the understanding lives

1 · Statement

the question

A concrete finance scenario with a tiny dataset you can see at a glance.

one slide

2 · Principles

the tools

The formulas involved and a short lab guide: what to compute, in what order.

one slide

3 · Hand Calc

pencil first

Work every number by hand in a table.
Small n , clean arithmetic, no calculator fear.

1–2 slides

4 · Claude Code

scale it up

A metaprompt: paste it into Claude Code and it builds the full lab on real data.

one slide

Hand first, machine second — you check the machine, never the other way around.

Where the Data Comes From

Public sources first — the prompts fetch or generate free data; CSV files are fallbacks only

Problem	Primary source (public · no signup)	Provided fallback file
P1 · Logistic	§ 5 in-script set (make_classification, fixed seed)	— generated in-script, always works
P2 · KNN	§ 5 in-script set + OpenML 'creditcard' (optional)	fml_w5_p2_applicants.csv (hand-calc)
P3 · Naive Bayes	§ 5 in-script set + OpenML 'creditcard' (optional)	fml_w5_p3_likelihoods.csv (hand-calc)
Real-data extension	sklearn fetch_openml('creditcard') — free, no key	—

Course rule: if data is free online (yfinance, FRED, OpenML) or generated in-script, the prompt uses it.

P1

PROBLEM 1 · LOGISTIC

From Week 1's Score to a Probability

Three applicants through the S-curve — and the threshold dial

The Old Score Learns to Say 'How Sure'

$z = 2 \cdot \text{Debt} - \text{Income}$ (Week 1). Push A, B and G through the sigmoid, then decide at two thresholds

Applicant	Income	Debt	$z = 2 \cdot \text{Debt} - \text{Income}$	Actually defaulted?
A	1.0	0.8	+0.6	Yes
B	2.0	0.5	-1.0	No
G	1.1	0.5	-0.1	Yes (Week 1's missed defaulter)

Helper (e-values)

- $e^{(-0.6)} \approx 0.549 \cdot e^{(1.0)} \approx 2.718 \cdot e^{(0.1)} \approx 1.105$.

Your deliverables

- $p = 1/(1+e^{(-z)})$ for A, B, G.
- Decisions at threshold 0.5 AND 0.3 — who flips, and why does the bank care?

The Tool: One Squash, One Dial

The score is Week 1's; the S-curve adds 'how sure'; the threshold adds 'how cautious'

$$p = \frac{1}{1+e^{-z}} \quad z = 2 \cdot \text{Debt} - 1 \cdot \text{Income} \quad (\text{the Week 1 score, now a probability})$$

Reading the squash

- $z = 0$ maps to $p = 0.5$ — the old decision line is the S-curve's pivot.
- Big positive $z \rightarrow p$ near 1; big negative $z \rightarrow p$ near 0.
- The output is a probability you can price, rank and explain.

Lab guide — per applicant

- Step 1: read z from the table.
- Step 2: $p = 1/(1 + e^{(-z)})$ using the helper values.
- Step 3: compare p to 0.5, then to 0.3 — write both verdicts.

Three Squashes, Two Verdicts Each

Watch borderline G flip when the bank turns the caution dial

Applicant	z	$e^{(-z)}$	$p = 1/(1+e^{(-z)})$	Verdict @ 0.5	Verdict @ 0.3
A	+0.6	0.549	$1/1.549 = 0.65$	DENY	DENY
B	-1.0	2.718	$1/3.718 = 0.27$	approve	approve
G	-0.1	1.105	$1/2.105 = 0.47$	approve (misses the defaulter)	DENY (caught!)

$$p_A = \frac{1}{1+e^{-0.6}} = 0.65 \quad p_B = \frac{1}{1+e^{1.0}} = 0.27 \quad p_G = \frac{1}{1+e^{0.1}} = 0.47$$

The Dial Caught What the Model Alone Missed

Same probabilities — a cautious threshold turned Week 1's FN into a catch

0.65

A — denied at any reasonable threshold

0.27

B — safely approved at both settings

0.47

G — approved at 0.5, DENIED at 0.3; the borderline decides

0.3

The dial: recall up, precision down — a business choice

Week 1 + Week 3, connected

- G was Week 1's expensive false negative.
- Lowering the threshold is the recall lever from Week 3.

The price of caution

- At 0.3, some good customers near $p = 0.35$ get denied too.
- The threshold encodes what a mistake costs the bank — not the model.

Scale It: The Full Threshold Sweep

Paste the metaprompt — Claude Code turns the dial across 20,000 transactions

[INPUT] Prompt · Claude Code

Build a single Colab-ready Python script that:

- 1) generates the class dataset:
`make_classification(n_samples=20000, n_features=20, n_informative=6, weights=[0.99, 0.01], random_state=42)` - fraud-like, ~1% positive;
- 2) splits train/test and fits LogisticRegression (`max_iter=1000`);
- 3) sweeps the decision threshold from 0.05 to 0.95 and prints a table: threshold | precision | recall | F1;
- 4) plots the precision-recall curve, marking the best-F1 threshold and the default 0.5;
- 5) three-sentence verdict: where would a bank that fears missed fraud set the dial, and what does it pay for that choice (Week 3 language)?

[OUTPUT] What to expect

best F1 near a threshold
well BELOW 0.5 (rare class)
recall climbs as the dial
drops; precision decays
PR curve: the honest picture
accuracy never shows

Your 3-applicant table,
swept across 19 dials.

P2

PROBLEM 2 • KNN

Let the Neighbors Vote

One query, six neighbors, three values of k — and a flip

A New Applicant and Six Old Files

Q(income 1.1, debt 0.6). Classify Q by majority vote with $k = 1, 3, 5$

Neighbor	Income	Debt	Defaulted?
A	1.0	0.7	Yes
B	1.3	0.5	No
C	0.8	0.8	Yes
D	1.5	0.7	No
E	1.2	0.3	No
F	0.7	0.4	Yes

Your deliverables

- Squared distance d^2 from Q to all six (no square roots needed).
- The sorted neighbor list.
- The vote and verdict at $k = 1, 3$, and 5.
- One sentence: what does the $k = 1$ vs $k = 3$ flip teach about noise?

Ranking by d^2 gives the same order as ranking by distance — skip the square roots.

The Tools: A Ruler and a Ballot Box

Measure, sort, count — KNN is arithmetic plus democracy

$$d^2(q, i) = (\text{inc}_i - \text{inc}_q)^2 + (\text{debt}_i - \text{debt}_q)^2 \quad \hat{y} = \text{majority vote of the } k \text{ nearest}$$

Why d^2 is enough

- The square root changes distances but never their ORDER.
- KNN only needs the order — so we vote straight from d^2 .
- (In real code: standardize features first, Week 4's lesson.)

Lab guide — three steps

- Step 1: $d^2 = (\Delta\text{income})^2 + (\Delta\text{debt})^2$ per neighbor.
- Step 2: sort ascending.
- Step 3: take the top k , count Yes vs No, majority wins.

Six Distances, Sorted, Then Three Votes

The nearest single file says Default — the neighborhood disagrees

Rank	Neighbor	d^2 to Q(1.1, 0.6)	Defaulted?	k=1	k=3	k=5
1	A	$(0.1)^2 + (0.1)^2 = 0.02$	Yes	✓	✓	✓
2	B	$(0.2)^2 + (0.1)^2 = 0.05$	No		✓	✓
3	E	$(0.1)^2 + (0.3)^2 = 0.10$	No		✓	✓
4	C	$(0.3)^2 + (0.2)^2 = 0.13$	Yes			✓
5	D	$(0.4)^2 + (0.1)^2 = 0.17$	No			✓
6	F	$(0.4)^2 + (0.2)^2 = 0.20$	Yes			—

$k = 1$: Default

$k = 3$: Safe

$k = 5$: Safe

— the single nearest neighbor was noise

Default → Safe → Safe as k Grows

k is a smoothing knob: 1 chases the nearest noise, 5 listens to the street

$k = 1$

Vote 1-0 → DEFAULT
— one neighbor rules

$k = 3$

Vote 1-2 → SAFE —
the flip; A was
outvoted

$k = 5$

Vote 2-3 → SAFE —
the verdict stabilizes

Week 1

Small k = Student A:
low bias, high
variance

Reading the flip

- A alone is the nearest file — but B and E right behind disagree.
- One noisy neighbor should not decide a loan.

How k is really chosen

- Sweep k on the VALIDATION set, pick the best F1.
- That sweep is exactly what the Claude prompt runs next.

Scale It: Sweep k on Twenty Thousand Rows

Paste the metaprompt — Claude Code finds the k the validation set prefers

[INPUT] Prompt · Claude Code

```
Build a single Colab-ready Python script that:  
1) generates the class dataset (make_classification,  
   20000 x 20, weights=[0.99, 0.01], random_state=42);  
2) splits train / validation / test (60/20/20) and  
   standardizes features with a sklearn Pipeline;  
3) sweeps KNN with k in [1, 3, 5, 9, 15, 25, 51] and  
   records validation F1 for each k;  
4) plots F1 vs k, picks the best k on VALIDATION, then  
   reports test precision / recall / F1 once, at that k;  
5) optional real-data run: repeat on  
   fetch_openml('creditcard') - free, no signup;  
6) three-sentence verdict: why k = 1 scores worst, in  
   Week 1 bias-variance language.
```

[OUTPUT] What to expect

```
k = 1: jumpy, weakest F1  
      (the Student A of KNN)  
F1 peaks at a moderate k,  
then blurs as k grows  
test scores quoted ONCE,  
at the validation winner
```

```
Your 6-file vote, held  
20,000 times.
```

P3

PROBLEM 3 · NAIVE BAYES

The Doctor Screens a Transaction

A 1% prior, three clues, one belief climbing to a flag

Three Clues Arrive, One After Another

A card transaction is foreign, at midnight, and unusually large. Fraud prior: 1%

Clue	$P(\text{clue} \mid \text{fraud})$	$P(\text{clue} \mid \text{normal})$
foreign transaction	0.60	0.05
midnight timestamp	0.50	0.10
unusually huge amount	0.70	0.10

Your deliverables

- The fraud and normal 'story scores' after each clue.
- The posterior $P(\text{fraud} \mid \text{clues so far})$ at each step.
- The verdict at threshold 0.5 — after which clue does it flip?

Naive assumption: multiply the clue probabilities as if the clues were independent.

The Tool: Two Competing Stories

Each clue multiplies both stories; the belief is the fraud story's share

$$P(\text{fraud} \mid \text{clues}) \propto P(\text{fraud}) \times \prod_j P(\text{clue}_j \mid \text{fraud}) \quad (\text{naive independence})$$

The two stories

- Fraud story starts at 0.01; normal story starts at 0.99.
- Each clue multiplies both by its likelihood under that story.
- Belief = fraud score $\div$ (fraud score + normal score).

Lab guide — per clue

- One multiplication per story, then one division.
- Three clues $\rightarrow$ three rows. Watch the belief climb.
- Flag when the belief crosses 0.5.

The Belief, Clue by Clue

Two strong clues were still not enough — base rates are stubborn

After...	Fraud score	Normal score	Belief P(fraud)	Verdict @ 0.5
(prior)	0.0100	0.9900	1.0%	pass
foreign	$0.01 \times 0.60 = 0.0060$	$0.99 \times 0.05 = 0.0495$	$0.0060 / 0.0555 = 10.8\%$	pass
+ midnight	$\times 0.50 = 0.0030$	$\times 0.10 = 0.00495$	$0.0030 / 0.00795 = 37.7\%$	pass — still!
+ huge amount	$\times 0.70 = 0.0021$	$\times 0.10 = 0.000495$	$0.0021 / 0.002595 = 80.9\%$	FLAG

1% $\xrightarrow{\text{foreign}}$ 10.8% $\xrightarrow{\text{midnight}}$ 37.7% $\xrightarrow{\text{huge amount}}$ 80.9%

1% → 10.8% → 37.7% → 80.9%

The doctor needed all three symptoms before naming the disease

× 38

Two strong clues lifted the belief 38-fold — and it STILL passed

80.9%

Only the third clue pushed it over the 0.5 line

1 / 1,000

The stubborn base rate every fraud system fights

3 mults

Total arithmetic — why NB screens millions per second

The base-rate lesson

- A rare class demands overwhelming evidence.
- This is accuracy's trap (Lab 1) seen from the probability side.

The 'naive' caveat

- Midnight and foreign clues likely overlap in reality.
- NB double-counts them a little — fast beats perfect at Stage 1.

Scale It: The Doctor Meets Real Data

Paste the metaprompt — Claude Code builds the bank's Stage-1 screen and tests it

[INPUT] Prompt · Claude Code

```
Build a single Colab-ready Python script that:  
1) generates the class dataset (make_classification,  
   20000 x 20, weights=[0.99, 0.01], random_state=42);  
2) fits GaussianNB and LogisticRegression on a train  
   split; on the test split prints for each:  
   precision, recall, F1, and wall-clock predict time;  
3) reproduces the hand calc: for one test row, print  
   the NB per-feature log-likelihood contributions,  
   showing how the belief accumulates clue by clue;  
4) optional real-data run: fetch_openml('creditcard')  
   - free, no signup - and repeat step 2;  
5) three-sentence verdict: where NB belongs in the  
   bank's 3-stage pipeline from class, and why its  
   'naive' flaw is acceptable there.
```

[OUTPUT] What to expect

```
NB: fastest predict time by  
   a wide margin  
Logistic: better F1, slower  
NB belief accumulation:  
   your 4-row table, printed  
   from real model internals  
  
Fast screen first, sharp  
score second - the pipeline.
```

Problem Set — Three Things Your Hands Now Know

You computed every number yourself; the code only repeats what you already understand

1

Logistic: the S-curve prices the old score

A 0.65 · B 0.27 · G 0.47 — and the 0.3 threshold caught Week 1's missed defaulter.

2

KNN: k is a smoothing knob

Default → Safe → Safe as k went 1 → 3 → 5 — one noisy neighbor should not decide a loan.

3

Naive Bayes: beliefs climb clue by clue

1% → 10.8% → 37.7% → 80.9% — base rates are stubborn; three multiplications screen millions.

Hand calculation first — then let Claude Code scale the same idea to real data.